AWS Certified AI Practitioner

AIF-C01 — Detailed Study Guide

EXAM OVERVIEW

Cost	$100 USD (50% voucher sometimes available)
Format	65 questions (50 scored + 15 unscored), 90 minutes
Pass score	700 / 1000 (compensatory model)
Validity	3 years
Mode	Online proctored (Pearson VUE) or test center
Domain weights	D1: 20% | D2: 24% | D3: 28% | D4: 14% | D5: 14%
Recommended prep	30–40 hrs (no AWS background) / 15–20 hrs (with AWS)

This guide covers all 5 domains of the AIF-C01 exam with detailed explanations, AWS service mappings, exam traps, and memory aids.

DOMAIN 1: Fundamentals of AI & ML (20%)

1.1 AI Hierarchy

AWS frames AI in concentric layers: **AI** is the broadest umbrella (any machine simulating human intelligence). **ML** is a subset that learns from data. **Deep Learning** is a subset of ML using neural networks (powers vision, speech, NLP). **Generative AI** is a subset of DL that creates new content. **Agentic AI** refers to systems that plan, reason, and take actions autonomously (e.g., Bedrock Agents).

> **Exam tip:** If a scenario says "creating new content" → GenAI. If "taking actions" or "multi-step workflow" → Agentic AI.

1.2 Types of Machine Learning

- **Supervised learning** uses labeled data for classification (categorical output) or regression (continuous output). AWS examples: Amazon Comprehend (sentiment), Rekognition (image labels), Fraud Detector.
- **Unsupervised learning** finds patterns in unlabeled data: clustering, anomaly detection. AWS examples: SageMaker K-Means, Random Cut Forest.
- **Reinforcement learning** learns by reward signals. AWS examples: DeepRacer, SageMaker RL.
- **Self-supervised learning** generates its own labels from unlabeled data. This is how foundation models are pre-trained.

1.3 SageMaker Inference Types (HIGH YIELD)

This topic appears repeatedly. Memorize all four:

Type	Latency	Payload	Use Case
Real-time	Low (ms)	Small	Persistent endpoint, always-on traffic (chatbots, fraud)
Serverless	Low-Med	Small	Intermittent/unpredictable traffic, scales to zero
Asynchronous	High	Up to 1 GB	Long processing (up to 1 hr), large payloads, queued
Batch Transform	Highest	Very large	Bulk offline jobs, no endpoint needed, scheduled

1.4 ML Lifecycle on AWS

Business problem framing → Data collection (S3, Kinesis) → Data preparation (SageMaker Data Wrangler, Glue) → Feature engineering (SageMaker Feature Store) → Model training (SageMaker Training Jobs) → Model evaluation → Deployment (SageMaker Endpoints) → Monitoring (Model Monitor, Clarify).

1.5 Key Terminology

- **Bias**: systematic error favoring certain outcomes. Detected by SageMaker Clarify.
- **Variance**: model sensitivity to training data fluctuations.
- **Overfitting**: high training accuracy, low test accuracy. Model memorized data.
- **Underfitting**: low accuracy on both training and test. Model too simple.
- **Hyperparameters**: set before training (learning rate, epochs). Tuned via SageMaker Automatic Model Tuning.
- **Parameters**: learned during training (weights, biases).

1.6 When NOT to Use ML

ML is not appropriate when rules are deterministic and well-known, when data is insufficient or low-quality, when 100% accuracy is required without human review, or when the cost of errors is unacceptably high.

DOMAIN 2: Fundamentals of Generative AI (24%)

2.1 Foundation Models (FMs)

A foundation model is a large model pre-trained on massive datasets via self-supervised learning, adaptable to many downstream tasks. FMs can be fine-tuned cheaper than training from scratch. On Bedrock you'll find Anthropic Claude, Amazon Titan, Amazon Nova, Meta Llama, Mistral, Cohere, AI21 Jurassic, and Stability AI.

2.2 LLM Building Blocks

- **Tokens**: text broken into chunks (~4 chars in English). Bedrock pricing is per input + output token.
- **Embeddings**: numerical vector representations of text/images. Used for semantic search and RAG. Services: Amazon Titan Embeddings, Cohere Embed.
- **Vector databases**: store embeddings for similarity search.
- **Context window**: max tokens a model processes at once.
- **Temperature**: controls randomness (0 = deterministic, 1 = creative).

2.3 Vector Databases on AWS

Memorize these — they appear in RAG questions:

- **Amazon OpenSearch Serverless** (default for Bedrock Knowledge Bases)
- **Amazon Aurora PostgreSQL** (with pgvector extension)
- **Amazon Neptune Analytics**
- **Amazon MemoryDB** (Redis-based, fastest)
- **Amazon DocumentDB**

TRAP: DynamoDB is NOT a vector database. Don't pick it for embedding storage.

2.4 Types of Generative Models

- **Transformers**: attention-based, used by most LLMs.
- **GANs**: Generative Adversarial Networks for image generation.
- **VAEs**: Variational Autoencoders for data generation.
- **Diffusion models**: image generation (Stable Diffusion).

2.5 Amazon Bedrock — The Star of the Exam

Bedrock is fully managed and serverless, providing single-API access to multiple FMs with no infrastructure to manage.

Key features:

- **Knowledge Bases**: managed RAG (connects FMs to your S3 data).
- **Agents**: multi-step task execution with FMs.
- **Guardrails**: filter harmful content, block topics, redact PII.
- **Model Evaluation**: compare models for your use case.
- **Custom Models**: fine-tune or do continued pre-training.
- **Provisioned Throughput**: reserved capacity for predictable workloads.
- **On-demand**: pay per token, no commitment.

2.6 Bedrock vs SageMaker JumpStart (CLASSIC TRAP)

Aspect	Amazon Bedrock	SageMaker JumpStart

Infrastructure	Fully managed, serverless	You deploy on YOUR SageMaker
Access	Single API call	Deploy endpoint first
Control	Limited, abstracted	Full (instances, hyperparams)
Use case	Fast, hands-off GenAI	Deep customization needed

Trigger phrases: "no infrastructure" → Bedrock. "Full control" / "custom infrastructure" → JumpStart.

2.7 Amazon Q Variants

- **Q Business**: enterprise GenAI assistant connecting to company data (SharePoint, Salesforce, S3).
- **Q Developer**: coding assistant in IDEs (was CodeWhisperer).
- **Q in QuickSight**: BI / analytics natural language queries.
- **Q in Connect**: contact center agent assistant.

2.8 GenAI Limitations

- **Hallucinations**: confident but false output. Mitigate with RAG, grounding, guardrails.
- **Knowledge cutoff**: model unaware of recent events. Mitigate with RAG.
- **Cost**: tokens add up; long contexts get expensive.
- **Latency**: bigger models = slower.
- **Bias**: reflects training data biases.
- **Non-determinism**: same input → different output (temperature > 0).

2.9 Bedrock Cost Optimization

- **On-demand**: variable / low traffic.
- **Provisioned throughput**: high, predictable volume — cheaper per token.
- **Batch inference**: 50% cheaper than on-demand for non-urgent jobs.
- **Smaller models**: cheaper, faster (e.g., Claude Haiku vs Opus).

DOMAIN 3: Applications of Foundation Models (28%)

This is the highest-weighted domain. Master customization methods and prompt engineering.

3.1 Prompt Engineering Techniques

- **Zero-shot**: ask the model to perform a task with no examples.
- **Few-shot / single-shot**: provide a few examples in the prompt.
- **Chain-of-thought (CoT)**: instruct the model to reason step by step. Improves math/logic dramatically.
- **Prompt templates**: reusable structures with placeholders (used in Knowledge Bases / Agents).

3.2 Prompt Engineering Best Practices

Be specific and clear. Provide context. Define output format. Use delimiters (XML, backticks). Specify role/persona. Set constraints (length, tone, language).

3.3 Prompt Risks & Mitigation

- **Prompt injection**: malicious user input that overrides system instructions.
- **Prompt leaking**: tricking the model into revealing its system prompt.
- **Jailbreaking**: bypassing safety guardrails.
- **Poisoning**: injecting harmful content during fine-tuning.
- **Hijacking**: taking control of model behavior.

Mitigation: Bedrock Guardrails, input validation, content filtering.

3.4 RAG (Retrieval Augmented Generation)

RAG grounds LLM responses in your private/recent data without retraining. The pipeline has 5 steps:

1. **Ingest** documents from a source (e.g., S3).
2. **Chunk** into smaller pieces.
3. **Embed** chunks into vectors (Titan/Cohere Embeddings).
4. **Store** in a vector DB (OpenSearch Serverless, etc.).
5. **Retrieve & Generate**: query → similarity search → top chunks injected into prompt → FM answers.

Bedrock Knowledge Bases handles all 5 steps automatically. Connects to S3, Confluence, Salesforce, SharePoint, web crawler.

3.5 FM Customization Methods (CRITICAL)

Method	Cost	Use When
Prompt engineering	Free	Simple tasks, no special data
RAG	Cheap	Private/recent data, reduce hallucinations
Fine-tuning (instruction)	Moderate	Specific style/format/tone, labeled examples
Domain adaptation FT	Moderate	Limited domain-specific labeled data
Continued pre-training	Expensive	Lots of UNLABELED domain data
Train from scratch	V. Expensive	Rarely needed

Decision rule: Style/format/tone → Fine-tuning. Knowledge/facts → RAG. Domain expertise + huge unlabeled corpus → Continued pre-training.

TRAP: AWS prefers RAG when data is "private", "recent", or "frequently updated". Don't pick fine-tuning unless the question emphasizes style/tone or labeled examples.

3.6 FM Evaluation Metrics

- **BLEU** → Translation quality.
- **ROUGE** → Summarization quality.
- **BERTScore** → Semantic similarity using BERT embeddings.
- **Perplexity** → How well a language model predicts a sample (lower = better).
- **Accuracy / Precision / Recall / F1** → Classification.
- **Human evaluation** → Gold standard, expensive.
- **Bedrock Model Evaluation** → Automated + human evaluation service for comparing FMs.

3.7 Inference Parameters

- **Temperature** (0–1): randomness. Low for factual, high for creative.
- **Top-p (nucleus)**: considers tokens summing to probability p.
- **Top-k**: considers top k most likely tokens.
- **Max tokens**: caps output length.
- **Stop sequences**: strings that halt generation.

3.8 Bedrock Agents

Agents enable LLMs to take actions, not just generate text. Components:

- **Foundation model**: the brain (e.g., Claude).
- **Instructions**: system prompt defining the agent's role.
- **Action Groups**: Lambda functions or OpenAPI schemas the agent can call.
- **Knowledge Bases**: optional RAG sources.
- **Memory**: optional, multi-turn context.

Pattern: "Multi-step task + take actions + use private data" → Bedrock Agents.

3.9 Choosing the Right FM

Selection factors: capability (reasoning, coding, multilingual, vision), cost per token, latency, context window, modalities (text vs multimodal), customization support, and regional availability.

3.10 Business Metrics for AI

Beyond model quality: task completion rate, user satisfaction (CSAT), cost per inference, time to resolution, conversion rate, error/hallucination rate, ROI.

DOMAIN 4: Guidelines for Responsible AI (14%)

4.1 Responsible AI Principles

- **Fairness**: AI treats all groups equitably.
- **Explainability**: can describe HOW a model reached a decision.
- **Privacy & Security**: protect data used in/by AI.
- **Safety**: AI behaves reliably and avoids harm.
- **Controllability**: humans can oversee, intervene, override.
- **Veracity & Robustness**: accurate outputs, resilient to attacks.
- **Governance**: policies, accountability, documentation.
- **Transparency**: openness about what the AI does.

4.2 Transparency vs Explainability (CLASSIC TRAP)

- **Transparency**: openness about the system — what data was used, what the model does, its limits, who built it. Documentation-focused.
- **Explainability**: ability to explain a SPECIFIC prediction — which features contributed and how.

Memory aid: Transparency = visible from outside (glass box). Explainability = the model explains its reasoning.

4.3 Bias and Fairness

Sources of bias: data bias, algorithmic bias, sampling bias, confirmation bias, measurement bias.

Mitigation:

- Use diverse, balanced, representative datasets.
- Audit across demographic slices.
- Use SageMaker Clarify to detect bias.
- Apply fairness metrics (demographic parity, equalized odds).
- Continuously monitor post-deployment.

4.4 AWS Tools for Responsible AI

SageMaker Clarify

Detects bias in data and trained models. Provides explainability via SHAP values (feature attribution). THE answer for bias detection and feature importance questions.

Bedrock Guardrails

Content filters (hate, violence, sexual, insults), denied topics, word filters, PII redaction, contextual grounding checks. Implements safety for GenAI.

AWS AI Service Cards

Documentation about AWS-managed AI services (Rekognition, Textract, Bedrock FMs). Cover intended use, limitations, performance, fairness. Implements transparency.

SageMaker Model Cards

YOU document YOUR custom models — training data, intended use, evaluation, ethical considerations. Customer equivalent of AI Service Cards.

Amazon Augmented AI (A2I)

Adds human review to AI predictions for low-confidence cases. Implements controllability and safety. Built-in workflows for Rekognition, Textract, plus custom.

SageMaker Model Monitor

Detects data drift and model drift in production. Alerts when performance degrades. Implements veracity and robustness.

4.5 Dataset Characteristics

Good datasets for responsible AI are: diverse, balanced, inclusive, curated, privacy-preserving, and representative of the deployment population.

4.6 Model Drift

- **Data drift**: input distribution changes (new product categories appear).
- **Concept drift**: relationship between inputs and outputs changes (what counts as fraud evolves).

Mitigation: SageMaker Model Monitor → detect drift → trigger retraining.

4.7 Legal & Ethical Considerations

GDPR (EU, right to explanation), CCPA (California), HIPAA (US healthcare), EU AI Act (risk-based regulation), copyright concerns for GenAI outputs and training data.

DOMAIN 5: Security, Compliance & Governance (14%)

5.1 IAM for AI Workloads

Apply least privilege. Use IAM roles (not long-term keys) for Bedrock, SageMaker, Q. Resource-based policies for S3 training data. Service Control Policies (SCPs) for org-wide guardrails. IAM Identity Center for Q Business user identity.

5.2 Data Protection

- **Encryption at rest**: AWS KMS for S3, EBS, SageMaker volumes.
- **Encryption in transit**: TLS for all API calls.
- **AWS KMS**: manages encryption keys; supports customer-managed keys (CMKs).
- **AWS Secrets Manager**: stores API keys, DB credentials.

KEY FACT: Customer prompts and outputs on Bedrock are NOT used to train base FMs. Your data stays yours.

5.3 Network Isolation

- **Amazon VPC**: isolate AI workloads in private networks.
- **VPC Endpoints (PrivateLink)**: access Bedrock, SageMaker, Q WITHOUT going over public internet.
- **SageMaker VPC mode**: run training/inference inside your VPC.

Pattern: "Without exposing data to public internet" → VPC endpoints / PrivateLink.

5.4 Auditing & Logging

- **AWS CloudTrail**: logs all API calls (who invoked which model, when).
- **Amazon CloudWatch**: metrics and logs (latency, errors, token usage).
- **AWS Config**: tracks configuration changes and compliance.
- **Bedrock Model Invocation Logging**: logs PROMPTS and RESPONSES to S3 / CloudWatch.

Pattern: "Audit prompts and responses" → Bedrock Model Invocation Logging (NOT just CloudTrail).

5.5 Compliance

- **AWS Artifact**: download compliance reports (SOC, ISO, HIPAA, PCI DSS, GDPR).
- **AWS Audit Manager**: automate compliance audits.
- Bedrock is HIPAA eligible, SOC compliant, GDPR-ready, PCI DSS supported.

5.6 Data Governance

- **AWS Glue Data Catalog**: metadata catalog.
- **AWS Lake Formation**: fine-grained access control on data lakes.
- **Amazon Macie**: detects PII / sensitive data in S3.
- **Amazon DataZone**: data governance and discovery across organization.

Pattern: "Detect PII in S3 training data" → Amazon Macie.

5.7 AI-Specific Security Risks

- **Prompt injection**: mitigate with Bedrock Guardrails.
- **Data leakage through prompts**: users pasting confidential info.
- **Model theft / extraction**: adversaries copying model via API.

- **Training data poisoning**: corrupted data biasing the model.
- **Membership inference**: determining if specific data was in training set.
- **PII exposure in outputs**: model regurgitating sensitive training data.

5.8 Bedrock Guardrails Capabilities

- **Content filters**: block hate, violence, sexual, insults.
- **Denied topics**: custom blocked topics (e.g., competitor discussion).
- **Word filters**: block specific words/profanity.
- **PII redaction**: mask/block SSNs, credit cards, etc.
- **Contextual grounding checks**: detect hallucinations vs source documents.

5.9 GenAI Security Scoping Matrix

AWS defines 5 scopes based on how you use FMs (more control = more responsibility):

1. **Scope 1: Consumer apps** (e.g., public ChatGPT) — least control.
2. **Scope 2: Enterprise apps** (GenAI features in SaaS).
3. **Scope 3: Pre-trained models** (Bedrock on-demand FMs).
4. **Scope 4: Fine-tuned models** (customizing FMs with your data).
5. **Scope 5: Self-trained models** — most control, most responsibility.

5.10 Shared Responsibility for AI

- **AWS**: security OF the cloud (infra, FM availability).
- **Customer**: security IN the cloud (IAM, prompts, fine-tuning data, guardrails config).

5.11 Cost Governance

AWS Budgets (alerts on spend), Cost Explorer (analysis), tags (cost by project), provisioned vs on-demand, Bedrock Batch (50% cheaper).

FINAL EXAM-DAY STRATEGY

Top Decision Patterns to Memorize

If question mentions...	Answer is likely...
"No infrastructure" / "fully managed" / "API"	Amazon Bedrock
"Full control" / "your own infrastructure"	SageMaker JumpStart
"Private data" / "company documents" / "no retraining"	RAG / Bedrock Knowledge Bases
"Specific style/tone" + "labeled examples"	Instruction-based fine-tuning
"Lots of UNLABELED domain data"	Continued pre-training
"Multi-step task" + "take actions" + "API calls"	Bedrock Agents
"Detect bias" / "feature importance"	SageMaker Clarify
"Low confidence" + "human review"	Amazon A2I
"Data drift" / "model degradation"	SageMaker Model Monitor
"Detect PII in S3"	Amazon Macie
"Block topics, filter PII in outputs"	Bedrock Guardrails
"Audit prompts AND responses"	Bedrock Model Invocation Logging
"Without public internet"	VPC Endpoints / PrivateLink
"Vector storage / semantic search"	OpenSearch Serverless (NOT DynamoDB)
"Translation evaluation"	BLEU
"Summarization evaluation"	ROUGE
"Code assistant in IDE"	Amazon Q Developer
"Contact center agent assist"	Amazon Q in Connect

Test-Taking Tips

1. Read each question TWICE. Identify keywords like "private", "no infrastructure", "unlabeled", "managed".
2. Eliminate clearly wrong answers first — usually 2 are obviously wrong.
3. There's NO penalty for guessing — answer every question, never leave blank.
4. Flag uncertain questions and review at the end if time permits.
5. Trust your first instinct — don't second-guess unless you spot a clear error.
6. Pace yourself: ~80 seconds per question on average.
7. Watch for distractors: AWS often includes plausible-but-wrong AWS services to test depth.

Recommended Final Prep

- Take the AWS Official Practice Exam (free on Skill Builder).
- Take 2–3 full-length mock exams from Tutorials Dojo / Stephane Maarek.
- Aim for 80%+ on practice exams before booking the real one.
- Review ALL wrong answers carefully — patterns repeat on the real exam.
- Read AWS AI Service Cards for Bedrock FMs (free, official).

Good luck — you've got this! ■